@ Narravula Muni Sai Teja

# Top AWS and DevOps Interview Questions

*This comprehensive document contains 200+ real-world interview questions and detailed answers compiled from recent interviews at top tech companies. Based on actual experiences shared by professionals in the field, covering AWS, Kubernetes, Docker, Terraform, Jenkins, CI/CD, and system troubleshooting scenarios*

## Pod Management & Troubleshooting

**Q: You deleted a deployment, but the pods are still running. How?**

**Answer:** The associated ReplicaSet wouldn't have been deleted. When a Deployment is paused or has finalizers preventing proper cleanup, the underlying ReplicaSet continues maintaining the desired pod count. This happens because Kubernetes follows a hierarchical ownership model where Deployments manage ReplicaSets, and ReplicaSets manage Pods. If the deletion cascade is interrupted by finalizers, admission webhooks, or controller issues, the child resources remain active. To resolve this, check for orphaned ReplicaSets using `kubectl get replicasets`, identify any finalizers blocking deletion, and manually clean up the remaining resources.

**Q: A pod with no memory limit defined is still getting OOMKilled. What could be the reason?**

**Answer:** Without a memory limit, the pod gets BestEffort Quality of Service and can overconsume node memory. When the node runs out of available memory, the kubelet's eviction manager kills BestEffort pods first to reclaim memory for system stability. This happens because pods without resource limits can potentially consume all available node memory, affecting system processes and other workloads. The Linux kernel's OOM killer also triggers when system memory is exhausted, prioritizing the termination of processes that consume the most memory without resource constraints.

## Autoscaling & Resource Management

**Q: A HPA is in place, but even under load, replicas remain at 1. What could cause this?**

**Answer:** The metrics server might not be reporting CPU or memory usage properly. Without valid metric data, the HPA controller cannot calculate resource utilization and won't initiate any scale-out actions. This commonly occurs when the metrics server is not deployed, not functioning correctly, or when pods lack resource requests which are required for HPA calculations. Additionally, if custom metrics are configured but the custom metrics API is unreachable, or if the HPA target metrics are set incorrectly, scaling actions will not trigger despite actual load increases.

## Deployment & Pod Creation Issues

**Q: A Deployment was rolled out, but pods are not created. What should be checked first?**

**Answer:** The Deployment might have failed admission due to validation or mutation webhooks. ValidatingAdmissionWebhooks or MutatingAdmissionWebhooks can block pod creation if the pod specification violates configured policies. Common causes include Pod Security Standards violations, resource quota exhaustion, or custom policy webhooks rejecting the pod configuration. Check the Events using `kubectl get events` and examine the ReplicaSet description to identify webhook-related errors that prevented pod creation from completing successfully.

## Network & Service Exposure

**Q: You exposed a pod via NodePort, but it's still unreachable externally. What's likely missing?**

**Answer:** The node-level firewall or cloud NACL might be blocking the assigned port. Even though Kubernetes successfully exposes the NodePort service, the underlying operating system or cloud infrastructure can still block external traffic. This includes firewall rules on the worker nodes, AWS security group restrictions, Network ACLs, or corporate firewall policies. The NodePort service creates the necessary iptables rules within Kubernetes, but external network policies must also permit traffic to the specific port range (typically 30000-32767) for external access to succeed.

## Container Image & Registry Management

**Q: A container image is updated in the registry, but pods are still pulling the old image. Why?**

**Answer:** The deployment might be using an immutable image digest or relying on the default ImagePullPolicy of IfNotPresent. When using specific image tags with IfNotPresent policy, nodes cache the image locally and avoid pulling newer versions with the same tag. This commonly occurs with 'latest' tags or when the image specification includes a specific

digest hash. To force image updates, either use ImagePullPolicy: Always, implement proper semantic versioning with unique tags, or restart the deployment to trigger fresh image pulls from the registry.

## Volume & Storage Management

**Q: You use emptyDir for temporary data, but after a pod restart, all the data is gone. Why?**

**Answer:** EmptyDir volumes have a lifecycle tied to the pod, not individual containers within the pod. When a pod is deleted, recreated, or rescheduled to a different node, the emptyDir volume is completely destroyed and recreated empty. This is the intended behavior for temporary storage. Container restarts within the same pod preserve emptyDir data, but pod-level events like eviction, deletion, or rescheduling result in complete data loss. For data persistence across pod restarts, use PersistentVolumes or other persistent storage solutions instead of emptyDir.

## System Performance & Debugging

**Q: A server's CPU is pegged at 100% but top shows no process consuming that much. How do you debug?**

**Answer:** This indicates kernel-level issues or hardware interrupt problems. Use system-level debugging tools like `htop` for detailed process information, check interrupt statistics with `/proc/interrupts` and `/proc/softirqs`, examine I/O wait patterns using `iostat`, and analyze kernel functions with `perf top`. The issue often stems from hardware interrupts, kernel modules, device driver problems, or I/O wait conditions that don't appear in standard process monitoring tools but consume CPU cycles at the kernel level.

**Q: A critical binary suddenly stops working after a package upgrade. Walk me through your root cause analysis.**

**Answer:** Begin with dependency analysis using `ldd` to identify missing or incompatible libraries, check package manager logs for recent changes, use `strace` to trace system calls and identify failure points, compare library versions before and after the upgrade, and examine configuration files for syntax or compatibility changes. The systematic approach involves isolating the change that caused the failure, testing rollback scenarios, and implementing proper staging procedures to prevent similar issues in production environments.

**Q: Disk I/O latency spikes randomly on a database host. What tools/commands do you use to pinpoint the issue?**

**Answer:** Use `iostat -x 1` for device-level I/O statistics, `iotop` to identify processes causing I/O load, `biosnoop` and `biolatency` from BCC tools for block I/O tracing, and `blktrace` for detailed block layer analysis. Check disk health with `smartctl`, examine filesystem fragmentation, monitor memory pressure causing swap activity, and analyze database-specific metrics like buffer pool hit ratios. The investigation focuses on identifying whether the latency originates from hardware, filesystem, or application-level issues.

## General & Day-to-Day Operations

**Q1: Explain your Day to Day activities**

**Answer:** As an AWS Cloud Engineer, my daily activities include monitoring infrastructure health using CloudWatch and Grafana dashboards, managing Kubernetes clusters through kubectl commands and Helm deployments, implementing Infrastructure as Code using Terraform for resource provisioning, maintaining CI/CD pipelines in Jenkins for automated application deployments, troubleshooting production issues through log analysis and

performance metrics, collaborating with development teams on containerization strategies, optimizing cloud costs through resource right-sizing, and ensuring security compliance through regular audits and policy updates.

**Q2: Do you have hands-on in Linux? If yes, Which Platform?**

**Answer:** Yes, I have extensive hands-on Linux experience primarily with Ubuntu Server (versions 18.04, 20.04, and 22.04) and Amazon Linux 2 on AWS EC2 instances. I've worked with CentOS and RHEL for enterprise applications. My experience includes system administration, shell scripting, process management, network configuration, security hardening, and package management across these distributions.

**Q3: What is the latest version of Ubuntu?**

**Answer:** Ubuntu 24.04 LTS (Noble Numbat) is the latest Long Term Support version released in April 2024. It provides 5 years of support and includes updated kernel version 6.8, improved security features, and enhanced container support. The LTS versions are preferred for production environments due to their stability and extended support lifecycle.

## AWS CI/CD Integration

**Q4: I have index.html on GitHub, I need to push it into AWS. The requirement is one load balancer is required and we provide index.html in GitHub repo and we provide username and token. How to achieve this?**

**Answer:** Create a CI/CD pipeline using AWS CodePipeline connected to GitHub repository using personal access token for authentication. The pipeline includes CodeBuild stage to build artifacts, store in S3 bucket, and CodeDeploy to deploy to EC2 instances behind an Application Load Balancer. Configure Auto Scaling Group with target groups, create security groups allowing HTTP/HTTPS traffic, and use Blue/Green deployment strategy for zero-downtime updates. Alternative approach uses GitHub Actions with AWS credentials to deploy directly to S3 for static hosting with CloudFront distribution.

## AWS Networking & Security

**Q5: What is dependencies resources for the IP?**

**Answer:** IP dependencies in AWS include VPC for network isolation, subnets for network segmentation, Internet Gateway for public internet access, Route Tables for traffic routing, Security Groups for instance-level firewall rules, Network ACLs for subnet-level filtering, NAT Gateway for private subnet outbound connectivity, Elastic IP for static public addressing, and DNS resolution through Route53. These resources form the network foundation required before assigning IP addresses to resources.

**Q6: What is the command to connect EC2 instance?**

**Answer:** The command is `ssh -i /path/to/private-key.pem ec2-user@public-ip-address` for Amazon Linux, or `ssh -i /path/to/private-key.pem ubuntu@public-ip-address` for Ubuntu instances. Alternative methods include AWS Systems Manager Session Manager using `aws ssm start-session --target instance-id` for secure shell access without SSH keys, or using EC2 Instance Connect for browser-based terminal access.

**Q7: How to implement Internet gateway?**

**Answer:** Create Internet Gateway using `aws ec2 create-internet-gateway`, attach it to VPC using `aws ec2 attach-internet-gateway --internet-gateway-id igw-xxx --vpc-id vpc-xxx`, modify route table to add route `0.0.0.0/0` pointing to Internet Gateway, and ensure subnets associated with this route table become public subnets. Security groups must allow inbound traffic on required ports for external access to work properly.

**Q8: How to configure internet gateway to route table?**

**Answer:** Add a route entry in the route table using `aws ec2 create-route --route-table-id rtb-xxx --destination-cidr-block 0.0.0.0/0 --gateway-id igw-xxx`. This creates a default route directing all non-local traffic to the Internet Gateway. Associate public subnets with this route table using `aws ec2 associate-route-table`

`--subnet-id subnet-xxx --route-table-id rtb-xxx` to enable internet connectivity for resources in those subnets.

## Linux System Administration

**Q9: What is the exact command to log in the instance and what kind of authentication will you use? Do you need password to provide or something else?**

**Answer:** Use SSH key-pair authentication with `ssh -i ~/.ssh/private-key.pem username@ip-address`. No password is required as AWS uses public-key cryptography for secure authentication. The private key must have correct permissions (600) using `chmod 600 private-key.pem`. Alternative authentication methods include AWS Systems Manager Session Manager for password-less access and EC2 Instance Connect for temporary SSH key injection.

**Q10: In which path will you create index.html in Server?**

**Answer:** For web servers, create index.html in `/var/www/html/` for Apache, `/usr/share/nginx/html/` for Nginx, or custom document root defined in server configuration. For development purposes, use `/home/username/public_html/` or project-specific directories. The exact path depends on web server type and configuration, but `/var/www/html/` is the standard location for most Linux web server installations.

## Essential Linux Commands

**Q11: Explain Most common linux Commands**

**Answer:** Essential commands include `ls` (list files), `cd` (change directory), `pwd` (current directory), `mkdir` (create directory), `rm` (delete files), `cp` (copy), `mv` (move), `find` (search

files), `grep` (search text), `ps` (processes), `top` (system monitor), `chmod` (permissions), `chown` (ownership), `df` (disk space), `du` (directory usage), `free` (memory), `netstat` (network connections), `systemctl` (service management), and `ssh` (secure shell).

**Q12: How to check one running process?**

**Answer:** Use `ps aux | grep process-name` to find specific process, `ps -ef | grep process-name` for detailed format, `pgrep process-name` to get process ID only, or `pidof process-name` for PID lookup. For real-time monitoring, use `top` or `htop` commands to see all running processes with resource usage statistics.

**Q13: How to list running process?**

**Answer:** Use `ps aux` to list all processes with detailed information, `ps -ef` for full format listing, `top` for real-time process monitoring, `htop` for enhanced interactive display, or `ps -ejH` for tree view showing parent-child relationships. Add filters like `ps aux | grep username` to show processes for specific users.

**Q14: You need to find one particular process id and how to kill that? Is it possible in single command?**

**Answer:** Yes, use `pkill process-name` to kill by name, `killall process-name` to kill all instances, or `kill $(pgrep process-name)` to combine find and kill operations. For forceful termination, use `pkill -9 process-name` or `kill -9 $(pgrep process-name)`. These single commands locate and terminate processes without manually finding PIDs first.

## System Monitoring

**Q15: How to check disk usage?**

**Answer:** Use `df -h` to check filesystem disk usage in human-readable format, `du -sh /path/` to check directory size, `du -h --max-depth=1 /path/` to see subdirectory sizes, or `lsblk` to display block device information. For detailed analysis, use `ncdu` for interactive disk usage explorer or `iostat` for I/O statistics.

**Q16: How to find free memory?**

**Answer:** Use `free -h` to display memory usage in human-readable format, `cat /proc/meminfo` for detailed memory information, `top` or `htop` for real-time memory monitoring, or `vmstat` for virtual memory statistics. The `free` command shows total, used, free, shared, buffer, and cache memory values.

## File Operations & Permissions

**Q17: How do you archive and compress directory in Linux?**

**Answer:** Use `tar -czf archive.tar.gz directory/` for gzip compression, `tar -cjf archive.tar.bz2 directory/` for bzip2 compression, or `zip -r archive.zip directory/` for ZIP format. To extract, use `tar -xzf archive.tar.gz`, `tar -xjf archive.tar.bz2`, or `unzip archive.zip` respectively.

**Q18: Chmod 755 means?**

**Answer:** Chmod 755 sets read, write, execute permissions for owner (7), and read, execute permissions for group and others (5 each). The octal notation breaks down as: owner (4+2+1=7), group (4+0+1=5), others (4+0+1=5), where 4=read, 2=write, 1=execute. This is common for executable files and directories.

**Q19: If I provide chmod 755 means, what exactly will happen?**

**Answer:** The file or directory will get read, write, and execute permissions for the owner, while group and other users get read and execute permissions only. For files, this makes them executable by everyone but writable only by owner. For directories, this allows the

owner full access while others can read contents and traverse the directory but cannot create or delete files within it.

**Q20: what is chown?**

**Answer:** Chown changes file or directory ownership. Syntax is `chown user:group filename` to change both user and group ownership, `chown user filename` to change only user ownership, or `chown :group filename` to change only group ownership. Use `-R` flag for recursive changes on directories. This command requires root privileges or ownership of the target file.

## User Management & Security

**Q21: How to list all ssh users in Linux?**

**Answer:** Check `/etc/passwd` file using `cat /etc/passwd | grep "/bin/bash"` to see users with shell access, `who` or `w` to see currently logged-in users, `last` to see recent login history, or `cat /etc/ssh/sshd_config | grep AllowUsers` to see SSH-allowed users if configured. Use `getent passwd` for comprehensive user list including LDAP users.

## Log Management

**Q22: If one apache server is running, I need to check logs? where do I check? In which directory?**

**Answer:** Apache logs are typically in `/var/log/apache2/` for Ubuntu/Debian or `/var/log/httpd/` for CentOS/RHEL. Main log files include `access.log` for request logs and `error.log` for error messages. Use `tail -f /var/log/apache2/access.log` for real-time monitoring or `grep ERROR /var/log/apache2/error.log` to filter specific entries.

**Q23: what kind of logs can see in /var/log?**

**Answer:** Common logs in `/var/log/` include `syslog` (system messages), `auth.log` (authentication), `kern.log` (kernel messages), `mail.log` (email server), `cron.log` (scheduled tasks), `apache2/` or `nginx/` (web server logs), `mysql/` (database logs), `ufw.log` (firewall), and `boot.log` (system boot). Each service typically creates its own log directory or file within `/var/log/`.

## Text Processing

**Q24: Purpose of Grep command?**

**Answer:** Grep searches for text patterns within files or input streams. It supports regular expressions for complex pattern matching, case-insensitive searches with `-i` flag, recursive directory searches with `-r`, line number display with `-n`, and inverse matching with `-v`. Common uses include log analysis, configuration file searches, and filtering command output.

**Q25: I need to grep keyword:linux. Expect the keyword linux, I need to list out all the lines from the linux.txt file. which flag will you used with grep?**

**Answer:** Use `grep -v linux linux.txt` to exclude lines containing "linux" and show all other lines. The `-v` flag inverts the match, displaying lines that do NOT contain the specified pattern. This is useful for filtering out unwanted entries while displaying everything else from the file.

## Task Scheduling

**Q26: Difference between "cron" and "at"?**

**Answer:** Cron schedules recurring tasks using crontab entries with time specifications (minute, hour, day, month, weekday). Jobs run repeatedly based on schedule. At schedules one-time tasks for specific future times using natural language or absolute time formats. Cron is for regular maintenance tasks, while at is for temporary, one-off scheduling needs.

**Q27: How to schedule activity in Linux and you need to run a script every 5 minutes?**

**Answer:** Use crontab with `crontab -e` to edit, then add entry `*/5 * * * * /path/to/script.sh` where `*/5` means every 5 minutes, other asterisks represent any hour, day, month, weekday. Ensure script has execute permissions with `chmod +x script.sh` and use absolute paths in cron entries for reliability.

## Git Version Control

**Q28: Explain Git pull and Git Fetch**

**Answer:** Git fetch downloads commits, files, and references from remote repository without merging them into current working branch, allowing review before integration. Git pull combines fetch and merge operations, automatically downloading and merging remote changes into current branch. Fetch is safer for collaborative environments as it allows inspection before integration, while pull provides convenience for straightforward updates.

**Q29: How do you resolve merge conflict?**

**Answer:** When merge conflict occurs, Git marks conflicted files with conflict markers `<<<<<<<`, `=======`, and `>>>>>>>`. Edit files to resolve conflicts by choosing desired changes, removing conflict markers, stage resolved files with `git add filename`, then complete merge with `git commit`. Use `git status` to see conflicted files and `git diff` to examine conflicts before resolution.

## Docker Containerization

**Q30: How to create custom image in Docker?**

**Answer:** Create Dockerfile with base image using `FROM ubuntu:20.04`, add instructions like `RUN`, `COPY`, `ADD`, `ENV`, `WORKDIR`, `EXPOSE`, and `CMD`. Build image using `docker build -t image-name:tag .` in directory containing Dockerfile. Best practices include using multi-stage builds, minimizing layers, using specific base image tags, and implementing proper security scanning before deployment.

**Q31: Elaborate Docker file.**

**Answer:** Dockerfile is a text file containing instructions to build Docker images. Key instructions include `FROM` (base image), `RUN` (execute commands), `COPY/ADD` (copy files), `WORKDIR` (set working directory), `ENV` (environment variables), `EXPOSE` (port exposure), `USER` (set user context), `VOLUME` (mount points), `ENTRYPOINT` (main command), and `CMD` (default parameters). Each instruction creates a new layer in the final image.

**Q32: What is the prerequisites of "docker build" command?**

**Answer:** Prerequisites include Docker daemon running, Dockerfile present in build context directory, proper syntax in Dockerfile, sufficient disk space for image layers, network connectivity for downloading base images and packages, appropriate permissions to read files in build context, and valid base image specification in FROM instruction.

**Q33: How do copy file from the container to host?**

**Answer:** Use `docker cp container-name:/path/to/file /host/path/destination` to copy from container to host, or `docker cp /host/path/file container-name:/path/to/destination` for reverse direction. Container can be running or stopped. Use `docker cp container-id:/path/to/file .` to copy to current host directory. This command preserves file permissions and ownership where possible.

**Q34: what if we put "COPY ." ? what will happen in background?**

**Answer:** `COPY .` copies all files and directories from build context (current directory where docker build is executed) into the container's current working directory set by WORKDIR. This includes all files, subdirectories, hidden files, but excludes files listed in `.dockerignore`. The operation creates a new layer in the image containing the copied content, which can significantly increase image size if build context contains large files.

## Docker Security

**Q: How do you secure Docker against kernel privilege escalation exploits?**

**Answer:** Use non-root users, drop all capabilities with `--cap-drop=ALL`, enable user namespaces, run with `--read-only` filesystem, use security profiles like AppArmor/SELinux, and regularly update images.

**Q: You want to run thousands of containers per host. What Linux tuning do you apply?**

**Answer:** Increase file descriptor limits, tune `fs.file-max`, optimize memory overcommit settings, increase network buffers, raise PID limits, and use resource limits per container.

**Q: You must ensure data at rest encryption for container volumes. How?**

**Answer:** Use encrypted EBS volumes, enable encryption in Docker storage driver, use LUKS for filesystem encryption, or implement application-level encryption within containers.

**Q: How do you handle multi-cloud Docker deployments with compliance restrictions?**

**Answer:** Use Terraform for consistent deployments, implement policy-as-code with OPA, maintain compliance documentation, use encrypted storage, and implement audit logging.

## Kubernetes Troubleshooting

**Q: The Kubernetes control plane becomes completely unreachable. How do you recover the cluster?**

**Answer:** Check etcd health, restore from backup if needed, verify certificates, restart control plane components (etcd → API server → controller → scheduler), check network connectivity.

**Q: etcd data corruption occurs, and your cluster won't start. What's your recovery plan?**

**Answer:** Stop all etcd processes, restore from latest backup using `etcdctl snapshot restore`, restart etcd cluster, then restart control plane components, verify cluster health.

**Q: A Pod is stuck in ImagePullBackOff. How do you troubleshoot?**

**Answer:** Check image name/tag, verify registry connectivity, check pull secrets, examine node disk space, verify image architecture matches node, check Docker daemon logs.

**Q: During high traffic, Ingress controller becomes a bottleneck. How do you fix this?**

**Answer:** Scale horizontally with HPA, use multiple ingress instances, optimize configuration, dedicate nodes for ingress, implement connection pooling, use external load balancers.

**Q: A StatefulSet upgrade failed and left half the pods in crashloop. How do you roll back safely?**

**Answer:** Pause rollout, identify last good revision, use `kubectl rollout undo`, rollback pod-by-pod from highest ordinal, verify data integrity after each step.

## Jenkins Optimization

**Q: How do you reduce Jenkins pipeline execution time in large projects?**

**Answer:** Use parallel stages, implement build caching, use multiple agents, optimize Docker layer caching, run incremental builds, create reusable pipeline libraries.

**Q: How do you manage pipelines for multiple microservices with interdependencies?**

**Answer:** Use multi-branch pipelines, implement upstream/downstream triggers, create shared libraries, use artifact repositories, implement service discovery for dependencies.

## Terraform

**Q: Rotate secrets used by Terraform-managed resources periodically. How?**

**Answer:** Use `time_rotating` resource, integrate with Vault/AWS Secrets Manager, implement Lambda for rotation, use external data sources for dynamic secrets.

**Q: How to destroy only a specific resource safely?**

**Answer:** Use `terraform destroy -target=resource_type.resource_name`, run plan first with `-destroy -target`, check dependencies with `terraform graph`.

**Q: Difference between count and for_each.**

**Answer:** Count uses numbers (resource, resource), problematic for dynamic lists. For_each uses keys, safer for dynamic resources, no recreation when removing middle items.[1]

**Q: How to handle Terraform provider versioning?**

**Answer:** Pin versions in `required_providers`, use lock files, test upgrades in staging first, maintain compatibility matrix, implement CI/CD version checks.

**Q: Terraform keeps recreating a resource unnecessarily. How do you debug it?**

**Answer:** Use `terraform plan -out=plan.out`, check for drift with `terraform refresh`, examine data source dependencies, use `ignore_changes` for external modifications.

**Q: How do you test Terraform code before production?**

**Answer:** Use Terratest for automation, run `terraform validate` for syntax, implement policy testing, use staging environments, run static analysis with tflint.

## SRE

**Q: How would you handle a high-severity outage in production?**

**Answer:** Activate incident response team, focus on restoration first, use runbooks, communicate with stakeholders, document actions, conduct post-incident review.

**Q: How to calculate error budgets? and Difference between SLI vs SLO vs SLA.**

**Answer:** SLI = metrics (uptime %), SLO = targets (99.9% uptime), SLA = contracts with penalties. Error Budget = (1 - SLO) × total requests.

**Q: How do you reduce toil in incident response?**

**Answer:** Automate common fixes, create runbooks, use chatops, implement self-healing systems, create better monitoring, reduce false alerts, use infrastructure as code.

**Q: How would you perform capacity planning for a rapidly growing service?**

**Answer:** Monitor utilization trends, use forecasting models, establish growth baselines, create capacity models, implement auto-scaling, plan for traffic spikes, coordinate with business teams.

## Jenkins & Deployment Strategies

### Environment Management

**Q31: Explain staging, production and testing environment.**

**Answer:** Testing environment is for developers to test code changes. Staging environment mirrors production for final testing and validation. Production environment serves live users. Each has isolated data, configurations, and access controls.

**Q32: What is a Blue Green Deployment?**

**Answer:** Blue-Green deployment uses two identical production environments. Blue is live, Green is updated. Traffic switches from Blue to Green after successful testing, enabling zero-downtime deployments and quick rollbacks.

**Q33: Explain Canary Deployment.**

**Answer:** Canary deployment releases new versions to a small percentage of users first. Gradually increases traffic to new version if metrics are good. Allows testing with real users while limiting risk exposure.

### Problem Solving & Scaling

**Q34: What is the biggest issue/hurdle/blocker you have faced, how did you fix it?**

**Answer:** In my LMS project, we faced memory issues with EKS pods getting OOMKilled. I analyzed resource usage patterns, implemented proper memory limits and requests, added HPA for scaling, and optimized application memory usage, reducing incidents by 90%.

**Q35: How do you scale your application?**

**Answer:** Use Horizontal Pod Autoscaler (HPA) for CPU/memory-based scaling, implement cluster autoscaling for nodes, use load balancers for traffic distribution, implement caching layers, optimize database queries, and use CDN for static content.

**Q36: How do you rollback if something fails?**

**Answer:** Use `kubectl rollout undo` for Kubernetes, implement blue-green deployments for instant switch-back, maintain previous image versions, use database backups for data rollback, implement automated rollback triggers based on health checks.

## Automation & Tools

**Q37: How do you automate Deployments?**

**Answer:** Create CI/CD pipelines with Jenkins, use GitOps with ArgoCD, implement Infrastructure as Code with Terraform, use Helm for Kubernetes deployments, integrate automated testing, and implement deployment triggers from Git commits.

**Q38: Which tools have you used for automating deployments?**

**Answer:** Jenkins for CI/CD pipelines, ArgoCD for GitOps, Helm for Kubernetes package management, Terraform for infrastructure, Docker for containerization, AWS CodePipeline/CodeDeploy, GitHub Actions for automation.

## Jenkins Pipeline & Configuration

**Q39: Have you written any Jenkins Pipeline from scratch?**

**Answer:** Yes, I've written declarative pipelines with stages for checkout, build, test, security scan, Docker build/push, and deployment. Used parallel execution, error handling, and integration with AWS services and Kubernetes.

**Q40: Mention some plugins you have used in Jenkins?**

**Answer:** AWS CLI, Docker Pipeline, Kubernetes CLI, Git, Blue Ocean, Pipeline, SonarQube Scanner, Slack Notification, Credentials Binding, ThinBackup, Role-based Authorization Strategy.

**Q41: What is CI/CD?**

**Answer:** Continuous Integration automatically builds and tests code changes. Continuous Deployment automatically releases validated changes to production. Enables faster, safer, and more reliable software delivery with reduced manual intervention.

## Jenkins Operations

**Q42: How can you create backup and copy files in Jenkins?**

**Answer:** Use ThinBackup plugin for scheduled backups, manually backup JENKINS_HOME directory, use `archiveArtifacts` in pipelines for build artifacts, copy files between jobs using `copyArtifacts` plugin, backup to external storage.

**Q43: Assume you have 2 Jenkins jobs, the first job completed successfully but the 2nd job failed. What will you do?**

**Answer:** Investigate logs of failed job, check dependencies and prerequisites, verify environment and permissions, restart failed job after fixing issues, implement job dependencies to prevent cascading failures, set up proper notifications.

**Q44: Can you tell me the process to integrate git with Jenkins?**

**Answer:** Install Git plugin, configure Git credentials in Jenkins, create job with Git SCM, specify repository URL and branch, set up webhooks in Git for automatic triggers, configure build triggers like polling or push events.

**Q45: Can you tell me what is DSL in Jenkins?**

**Answer:** Job DSL (Domain Specific Language) allows creating Jenkins jobs using Groovy-based code instead of UI. Enables Infrastructure as Code for Jenkins, version control of job configurations, and automated job creation/management.

## Jenkins Administration

**Q46: Why we use Jenkins instead of Bamboo? Difference between them?**

**Answer:** Jenkins is open-source with large community support, extensive plugin ecosystem, free to use. Bamboo requires licensing, has limited plugins, better Atlassian integration. Jenkins offers more flexibility and customization options.

**Q47: How to deploy a custom build of a code plugin to Jenkins?**

**Answer:** Build plugin using Maven with `mvn hpi:hpi`, upload .hpi file through Manage Jenkins → Manage Plugins → Advanced → Upload Plugin, restart Jenkins, or deploy to update center for distribution.

**Q48: What is the use of Jenkins home directory?**

**Answer:** JENKINS_HOME stores all Jenkins data including job configurations, build history, workspace, plugins, system settings, credentials, and logs. Essential for backup and migration operations.

**Q49: How do take backup of Jenkins Jobs?**

**Answer:** Use ThinBackup plugin for automated backups, manually copy job config.xml files from jobs directory, export job configurations using CLI, use Backup plugin, store backups in external location for safety.

**Q50: What is the way to configure Jenkins nodes agent with Jenkins master?**

**Answer:** Add new node in Manage Jenkins → Manage Nodes, configure connection method (SSH, JNLP), set labels and executors, install Java on agent, establish connection, verify agent appears online and ready for builds.

## DevOps & Linux Fundamentals

**Q1: What is DevOps Lifecycle?**

**Answer:** Plan → Code → Build → Test → Release → Deploy → Operate → Monitor. Continuous cycle focusing on collaboration between development and operations teams, automation, and continuous improvement for faster, reliable software delivery.

**Q2: Have you used any Linux Flavors, if yes, which one?**

**Answer:** Yes, I've worked with Ubuntu Server (18.04, 20.04, 22.04), Amazon Linux 2 on AWS EC2, and CentOS/RHEL for enterprise applications. Ubuntu for development, Amazon Linux for AWS workloads.

**Q3: What is the command to change the ownership and permission of a file or directory in Linux?**

**Answer:** `chown user:group filename` for ownership, `chmod 755 filename` for permissions. Use `-R` for recursive changes on directories. Example: `sudo chown nginx:nginx /var/www/html -R`

**Q4: How do you manage and view running processes in Linux?**

**Answer:** Use `ps aux` for all processes, `top` or `htop` for real-time monitoring, `pgrep process-name` to find PIDs, `kill PID` or `pkill process-name` to terminate processes.

## Networking Fundamentals

**Q5: What is SSH?**

**Answer:** Secure Shell - encrypted protocol for remote login and command execution. Uses public-key cryptography for authentication. Default port 22. Command: `ssh user@hostname`

**Q6: What is DNS (Domain Name System), and how does it work?**

**Answer:** DNS translates domain names to IP addresses. Process: Query → Local Cache → ISP DNS → Root servers → TLD servers → Authoritative servers → Response with IP address.

**Q7: What is NAT(Network Address Translation), and why is it used?**

**Answer:** NAT translates private IP addresses to public IPs for internet access. Conserves

public IP addresses, provides security by hiding internal network structure, enables multiple devices to share single public IP.

**Q8: Explain the difference between TCP and UDP Protocols?**

**Answer:** TCP is connection-oriented, reliable, ordered delivery with error checking. UDP is connectionless, faster, no delivery guarantee. TCP for web browsing, UDP for streaming, gaming, DNS queries.

## Version Control

**Q9: What os Git and how do we use it in DevOps?**

**Answer:** Git is distributed version control system. In DevOps: source code management, branching strategies, CI/CD triggers, collaboration, code reviews, rollback capabilities, infrastructure as code versioning.

**Q10: Explain the workflow of how to push the code from a local machine?**

**Answer:** `git add .` → `git commit -m "message"` → `git push origin branch-name`. Full flow: modify code → stage changes → commit locally → push to remote repository.

**Q11: How do you revert a commit that you made in your repository?**

**Answer:** `git revert commit-hash` creates new commit undoing changes, `git reset --hard commit-hash` removes commits completely. Use revert for shared repositories, reset for local changes only.

**Q12: What is a Branch in a Repository?**

**Answer:** Independent line of development in Git. Allows parallel feature development, isolates changes, enables collaboration without conflicts. Main branches: master/main, develop, feature branches.

## Cloud & AWS

**Q13: What cloud you are familiar with?**

**Answer:** AWS is my primary cloud platform with 2+ years experience. Familiar with core

services: EC2, S3, RDS, Lambda, VPC, EKS, ECS, CloudWatch, IAM. Some exposure to Azure and Google Cloud.

**Q14: What is VPC in cloud?**

**Answer:** Virtual Private Cloud - isolated network environment in cloud. Provides network segmentation, security control, custom IP addressing, subnets, route tables, gateways for resource isolation.

**Q15: What is the difference between a Private and Public Subnet and what differentiates it?**

**Answer:** Public subnet has internet gateway route (0.0.0.0/0 → IGW), allows direct internet access. Private subnet uses NAT gateway for outbound only. Route table configuration differentiates them.

**Q16: What is difference between reserved instance and spot instances?**

**Answer:** Reserved instances offer discounts for committed usage (1-3 years), guaranteed capacity. Spot instances use spare capacity at up to 90% discount but can be terminated anytime.

## Infrastructure as Code

**Q17: What is CloudFormation in AWS?**

**Answer:** AWS Infrastructure as Code service using JSON/YAML templates. Creates, updates, deletes AWS resources declaratively. Provides version control, rollback capabilities, and consistent deployments.

**Q18: What are the popular Infra as Coding tools have you used?**

**Answer:** Terraform for multi-cloud infrastructure, AWS CloudFormation for AWS-specific resources, Ansible for configuration management, Helm for Kubernetes package management.

**Q19: What is the difference between Terraform & Ansible?**

**Answer:** Terraform provisions infrastructure (immutable), Ansible configures systems

(mutable). Terraform is declarative for infrastructure, Ansible is procedural for configuration management and deployment.

**Q20: What is a playbook in Ansible?**

**Answer:** YAML file containing tasks, plays, and configurations to automate system setup. Defines desired state, includes hosts, tasks, variables, handlers for configuration management.

**Q21: What is a state file in Terraform?**

**Answer:** JSON file tracking actual infrastructure state vs configuration. Contains resource metadata, dependencies, current state. Essential for planning changes and managing infrastructure lifecycle.

**Q22: What is Terraform Remote State backend?**

**Answer:** Stores state file remotely (S3, Azure Storage) instead of locally. Enables team collaboration, state locking, backup, and prevents conflicts in multi-user environments.

## Containerization

**Q23: What is the Difference between Virtualisation and containerisation?**

**Answer:** Virtualization uses hypervisor to create full VMs with separate OS. Containerization shares host OS kernel, lighter weight, faster startup, better resource utilization.

**Q24: What problem does Docker solve?**

**Answer:** Solves "works on my machine" problem. Provides consistent environments, application portability, resource efficiency, simplified deployment, microservices architecture support.

**Q25: What is a Dockerfile and why do we use it?**

**Answer:** Text file with instructions to build Docker images. Automates image creation,

ensures reproducibility, version control for container configurations, standardizes deployment process.

**Q26: Explain the workflow of a Docker container, creation/working?**

**Answer:** Write Dockerfile → `docker build` creates image → `docker run` creates container → Container runs application → `docker stop` stops container → `docker rm` removes container.

**Q27: How do you manage multiple Containers?**

**Answer:** Use Docker Compose for local multi-container applications, Kubernetes for production orchestration, Docker Swarm for simple clustering, or managed services like AWS ECS/EKS.

## CI/CD

**Q28: What is CI and CD in CICD?**

**Answer:** Continuous Integration automatically builds and tests code changes. Continuous Deployment automatically releases validated changes to production. Enables faster, safer software delivery.

**Q29: What CICD tools have you used in the past?**

**Answer:** Jenkins for pipelines, GitHub Actions for automation, AWS CodePipeline/CodeBuild for AWS integration, ArgoCD for GitOps, GitLab CI for integrated development workflow.

**Q30: How will you create a CICD pipeline to update the website or app on every commit to a particular branch?**

**Answer:** Create pipeline with source stage (GitHub webhook), build stage (compile/test), deploy stage (to target environment). Configure branch-specific triggers, automated testing, and deployment approval gates.

**Q31: Explain staging, production and testing environment.**

**Answer:** Testing environment for developers to test features. Staging environment mirrors production for final validation. Production environment serves live users. Each isolated with different configurations and data.

# Advanced Kubernetes Interview Questions

## Pod Lifecycle & Containers

**Q1: If you have a Pod with initContainers that fail, but the main container has restartPolicy: Never, what happens to the Pod status?**

**Answer:** The Pod stays in **Init** status and won't start main containers. InitContainers must complete successfully before main containers start. With restartPolicy: Never, failed initContainers won't restart, keeping Pod in pending/init state permanently.

**Q2: When using a StatefulSet with 3 replicas and you delete replica-1, will replica-2 and replica-3 be renamed to maintain sequential ordering?**

**Answer: No**, they won't be renamed. StatefulSet maintains stable identities. If replica-1 is deleted, replica-2 and replica-3 keep their names. When scaling up later, a new replica-1 will be created with the same identity.

**Q3: Can a DaemonSet Pod be scheduled on a master node that has NoSchedule taint without explicitly adding tolerations?**

**Answer: Yes**, DaemonSets automatically include tolerations for common taints including `node-role.kubernetes.io/master:NoSchedule`. This allows system pods like kube-proxy, CNI, and monitoring agents to run on master nodes.

## Deployment & Rolling Updates

**Q4: If you update a Deployment's image while a rolling update is in progress, will K8s wait for the current rollout to complete or start a new one immediately?**

**Answer:** Kubernetes **starts a new rollout immediately**. It cancels the current rolling update and begins a new one with the updated image. The deployment controller ensures only the latest revision is being rolled out.

**Q5: When a node becomes NotReady, how long does it take for Pods to be evicted, and can this be controlled per Pod?**

**Answer:** Default is **5 minutes** (300 seconds). Controlled by `tolerationSeconds` in pod spec. Set `tolerations` with `tolerationSeconds: 60` for faster eviction or remove it for pods that should never be evicted due to node issues.

## Resource Sharing & Access

**Q6: Is it possible for a Pod to have multiple containers sharing the same port on localhost, and what happens if they try to bind simultaneously?**

**Answer: No**, only one container can bind to a specific port. If multiple containers try to bind to the same port, the second one will fail with "address already in use" error. Use different ports or implement port sharing at application level.

**Q7: If you create a PVC with ReadWriteOnce access mode, can multiple Pods on the same node access it simultaneously?**

**Answer: Yes**, ReadWriteOnce means one **node** can mount the volume, but multiple pods on that same node can access it simultaneously. ReadWriteOnce is node-level, not pod-level restriction.

## Autoscaling & Metrics

**Q8: When using Horizontal Pod Autoscaler with custom metrics, what happens if the metrics server becomes unavailable during high load?**

**Answer:** HPA stops scaling decisions and maintains current replica count. It cannot scale up or down without metric data. Implement monitoring for metrics server availability and consider multiple metric sources for redundancy.

**Q9: Can you run kubectl port-forward to a Pod that's in CrashLoopBackOff state, and will it work?**

**Answer: No**, port-forward requires the pod to be in Running state. CrashLoopBackOff means containers are failing to start, so there's no running process to forward traffic to. Fix the underlying issue first.

## Security & Service Accounts

**Q10: If a ServiceAccount is deleted while Pods using it are still running, what happens to the mounted tokens and API access?**

**Answer:** Existing pods keep working with cached tokens until they expire (default 1 hour). API access continues until token expiration. New pods cannot be created with the deleted ServiceAccount. Existing pods lose API access when tokens expire.

**Q11: When using anti-affinity rules, is it possible to create a "deadlock" where no new Pods can be scheduled?**

**Answer: Yes**, strict anti-affinity with insufficient nodes can prevent scheduling. If you have 2 nodes but require 3 pods with anti-affinity, the third pod will remain Pending. Use preferred anti-affinity or add more nodes.

## Jobs & Workload Management

**Q12: If you have a Job with parallelism: 3 and one Pod fails with restartPolicy: Never, will the Job create a replacement Pod?**

**Answer: Yes**, Job controller creates replacement pods for failed ones when restartPolicy: Never. It continues until `completions` number of pods succeed or `backoffLimit` failures are reached.

**Q13: Can a Pod's resource requests be modified after creation, and what's the difference between requests and limits during OOM scenarios?**

**Answer: No**, resource requests/limits cannot be modified after pod creation. During OOM: requests determine scheduling and QoS class, limits trigger OOMKill. Pods exceeding limits get killed first, then by QoS class (BestEffort → Burstable → Guaranteed)